

Individual Contribution Report

TruPharma Clinical Intelligence Platform

Submitted by: **Nithin Reddy** | GitHub: **reddy-nithin**

Repository: **reddy-nithin/TruPharma-Clinical-Intelligence**

1. Description of Personal Contributions

My contributions to the TruPharma Clinical Intelligence project focused on two major technical pillars: **advanced prompt engineering for the RAG-based query engine** and **full containerisation of the application using Docker**. Together these changes improved both the quality of AI-generated clinical responses and the operational readiness of the platform for deployment.

1a. Prompt Engineering Overhaul

I completely rewrote the core RAG prompt pipeline to produce adaptive, evidence-grounded responses. Key technical work included:

- Redesigned `_RAG_SYSTEM` with adaptive format guidance keyed to four distinct query types: simple lookup, drug-drug interaction, comparison, and mechanism-of-action.
- Implemented a three-tier citation hierarchy: **[Evidence N]** for retrieved passages, **[KG]** for knowledge-graph facts, and **[FAERS]** for FDA adverse-event data.
- Added FDA-primary conflict resolution logic, graceful off-topic redirects that name *TruPharma*, and a treatment-request orientation pattern including a medical disclaimer.
- Fixed a critical scope-gate bug: off-topic or no-drug queries now reach the Gemini LLM for a warm redirect instead of returning a hard-coded "Not enough evidence" error.
- Restructured `_build_prompt()` with explicit section separators, a query-type hint field, and in-prompt conversation history (replacing the previous external-prepend approach).
- Enhanced `_ANALYSIS_PROMPT` with a `context_clues` field, mechanism-intent detection, brand-to-generic normalisation, and a negative example for off-topic queries.
- Updated `_parse_response()` and `_regex_fallback()` to handle all new structured fields.

1b. Docker Containerisation

I containerised the entire TruPharma stack to enable one-command local development and cloud deployment. Work completed:

- Authored a production-grade **Dockerfile** based on Python 3.11.12-slim with a non-root user and a `/healthcheck` endpoint.
- Created **docker-compose.yml** with a single app service, volume mounts for `data/`, `assets/`, and `logs/`, and port mapping to **localhost:8501**.
- Added **.dockerignore** to exclude secrets, large data directories, and vendor files from the build context.
- Updated **.gitignore** to cover `.secrets/` and `docker-compose.override.yml`.

- Enforced security best-practices: non-root container user, `.env` injected at runtime (never baked into the image), GCP credentials mounted read-only, and no secrets stored in Docker layers.

2. GitHub Commits & Repository Evidence

Commit 1 — Prompt Engineering Overhaul

Commit SHA	3da11e95f3496e6ba70ea79f7ca3d0e50d7a3889
Date	19 March 2026
Lines added	+736 lines / -66 lines (802 total changes)
Files changed	4 files (engine.py, query_analyzer.py, test_prompt_eval.py, prompt_eval_template.md)
Summary	feat: prompt engineering overhaul for adaptive, evidence-grounded responses

Files modified in this commit:

```
src/rag/engine.py +147 / -50 (core RAG prompt rewrite)
src/rag/query_analyzer.py +45 / -16 (analysis-prompt improvements)
tests/test_prompt_eval.py +327 (8-category automated eval suite)
reports/prompt_eval_template.md +217 (manual comparison template & rubric)
```

Commit 2 — Docker Containerisation

Commit SHA	d60369af14f33a71eced2e62782deb9018b85936
Date	31 March 2026
Lines added	+181 lines (net new files)
Files changed	4 files (Dockerfile, docker-compose.yml, .dockerignore, .gitignore)
Summary	feat: Dockerize TruPharma — full containerization with docker-compose

Files added / modified in this commit:

```
Dockerfile +74 (Python 3.11-slim, non-root, healthcheck)
docker-compose.yml +37 (service definition, volumes, ports)
.dockerignore +66 (build-context exclusions)
.gitignore +4 (secrets & override exclusions)
```

3. Tools Used

Claude Sonnet 4.6	AI pair-programming assistant (co-authored commit 1 as noted in commit message)
GitHub / Git	Version control, commit history, and evidence of contributions
Python 3.11	Primary language for RAG engine, query analyser, and test suite
Google Gemini LLM	Underlying LLM for generating clinical responses via the RAG pipeline
Docker / Docker Compose	Application containerisation and local orchestration
ReportLab (PDF generation)	Used to produce this contribution report document
Streamlit	Frontend web framework serving TruPharma on port 8501

4. Reflection on Improvements Made

What improved in the system

The prompt engineering overhaul produced a measurably more coherent and trustworthy clinical assistant. Before the changes, the RAG engine would return a blunt error message for any query that did not match an exact drug name. After the rewrite, the system gracefully handles ambiguous, off-topic, or broad treatment queries by redirecting the user with an explanatory response instead of a dead end. The three-tier citation hierarchy also makes it transparent to the user exactly where each piece of information came from — RAG retrieval, the knowledge graph, or FDA FAERS pharmacovigilance data.

What I learned

Working on adaptive prompt design taught me how sensitive LLM output quality is to system-prompt structure. Small changes — such as placing conversation history inside the prompt rather than prepending it externally, or adding an explicit query-type hint — produced noticeably better results. I also deepened my understanding of structured output parsing: relying solely on regex fallbacks was fragile; building a schema-aware parser for the new fields made the pipeline more robust.

What the Docker work added

Containerising the project removed the "works on my machine" problem entirely. Anyone with Docker installed can now spin up the full TruPharma stack with a single `docker compose up -d` command. I also gained hands-on practice with production security patterns: running as a non-root user, excluding secrets from image layers, and mounting credentials read-only at runtime are habits I will carry into future projects.

Challenges encountered

The biggest challenge in the prompt work was the scope-gate logic: the original code had an early-return path that bypassed the LLM entirely for off-topic queries, so my new system-prompt instructions were never even sent. Diagnosing and fixing that control-flow bug

required careful tracing through the engine. For Docker, the main challenge was ensuring that secrets (API keys, GCP service-account JSON) were never baked into the image while still being available at runtime — solved by combining *.dockerignore*, *.env* injection, and read-only volume mounts.

All work was committed under the GitHub account **reddy-nithin** and is verifiable via the commit SHAs listed in Section 2 of this report.